

03_CONCURRENCY: DEADLOCK AND STARVATION PROBLEMS, MORE ON MUTUAL EXCLUSION

1. DEADLOCK

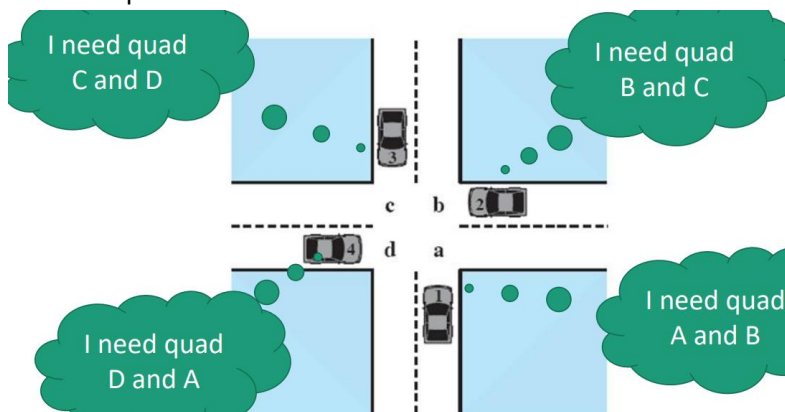
Deadlock: il blocking permanente di un insieme di processi che competono per risorse del sistema o comunicano fra di loro.

Un insieme di processi è in deadlock quando ogni processo nell'insieme è blocked aspettando un evento che può essere scaturito solamente da un altro processo nell'insieme che è blocked.

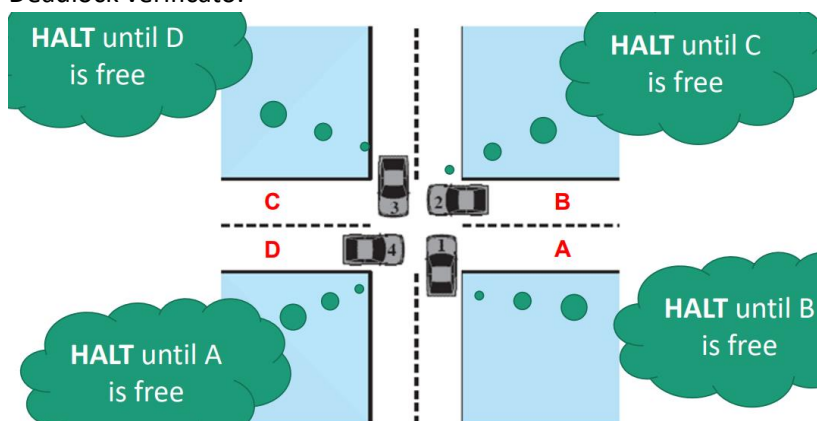
Il deadlock è permanente, non esiste una soluzione efficiente.

es. reale

Deadlock potenziale:



Deadlock verificato:



Deadlock e busy waiting sono due cose indipendenti, non collegate.

Ci sono diverse categorie di risorse:

- **Reusable resource:** può essere usato in maniera sicura solamente da un processo per volta, e non è esaurito da tale utilizzo.
Ad esempio: processors, I/O channels, main and secondary memory, devices, and structures such as files, databases, and semaphores.
- **Consumable resource:** risorsa che può essere creata (produced) e distrutta (consumed).
Ad esempio: interrupts, signals, messages, and information in I/O buffers.

es. reusable resources (due processi in competizione)

Process P		Process Q	
Step	Action	Step	Action
P ₀	Request (D)	q ₀	Request (T)
P ₁	Lock (D)	q ₁	Lock (T)
P ₂	Request (T)	q ₂	Request (D)
P ₃	Lock (T)	q ₃	Lock (D)
P ₄	Perform function	q ₄	Perform function
P ₅	Unlock (D)	q ₅	Unlock (T)
P ₆	Unlock (T)	q ₆	Unlock (D)

In P aspetta T che è bloccato da Q, e Q aspetta D che è bloccato da P

es. memory request

È disponibile spazio per l'allocazione di 200 kbytes, e occorre la seguente sequenza di eventi:

P1	P2
...	...
Request 80 Kbytes;	Request 70 Kbytes;
...	...
Request 60 Kbytes;	Request 80 Kbytes;

Occorre deadlock se entrambi i processi arrivano alla loro seconda richiesta

es. deadlock con consumable resources

consideriamo un paio di processi, ciascun processo cerca di ricevere un messaggio dall'altro processo e poi invia un messaggio all'altro

P1	P2
...	...
Receive (P2);	Receive (P1);
...	...
Send (P2, M1);	Send (P1, M2);

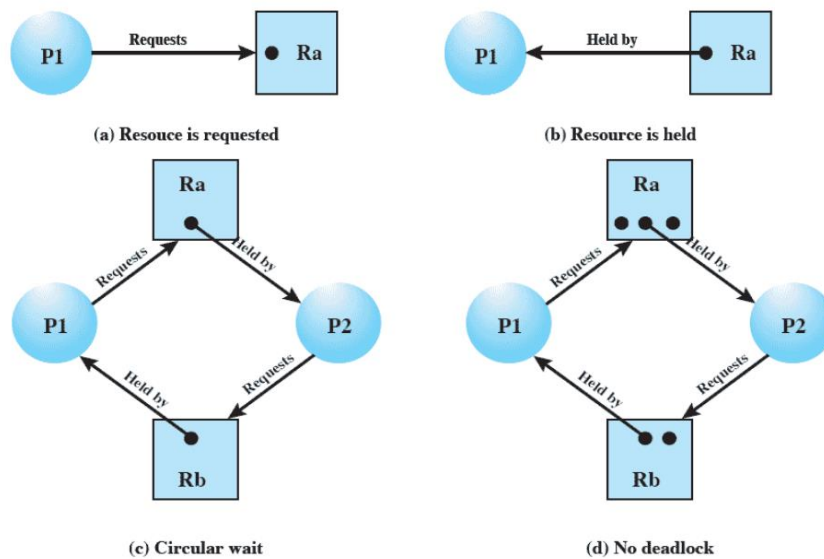
Se la receive è blocking allora si verifica deadlock

Il deadlock si verifica quando ci sono le seguenti 4 condizioni:

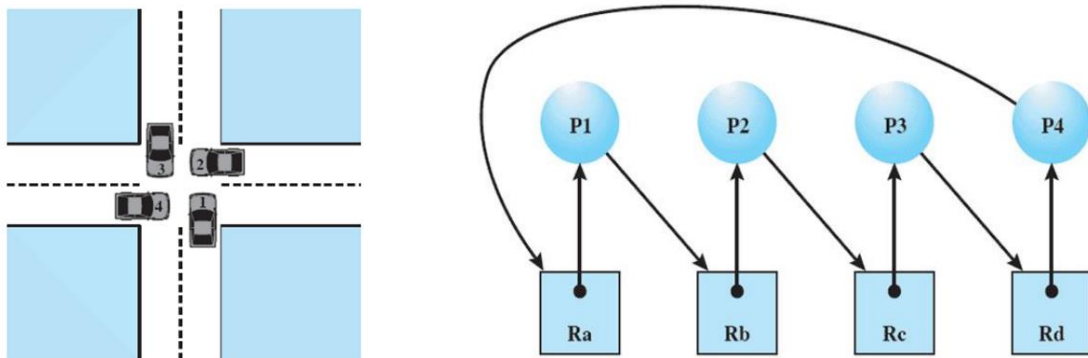
- **Mutual Exclusion**, solo un processo per volta può usare una risorsa
- **Hold-and-Wait**, un processo potrebbe trattenere risorse allocate mentre aspetta che altri processi facciano qualcosa
- **No Pre-emption**, nessuna risorsa può essere rimossa forzatamente da un processo che la sta trattenendo
- **Circular Wait**, esiste una catena chiusa di processi, tale che ogni processo trattiene almeno una risorsa necessaria al processo successivo nella catena

Le prime 3 condizioni sono necessarie ma non sufficienti.

Grafi di resource allocation:



es.



2. GESTIONE DEADLOCK: PREVENTION, AVOIDANCE, DETECTION

Ci sono 3 principali approcci per gestire i deadlock:

- **Prevent Deadlock:** si adotta una politica che elimina una delle condizioni necessarie per deadlock.

Strategia di **deadlock prevention**: si progetta un sistema in modo che sia esclusa la possibilità di deadlock. Ci sono due metodi principali:

- **Indiretto:** si evita la presenza di una delle 3 condizioni necessarie per il deadlock.
- **Diretto:** si evita la presenza di una circular wait

Prevenzione della deadlock condition:

- Mutual exclusion, se l'accesso alla risorsa necessita mutua esclusione, allora dev'essere supportata dall'OS.

Di solito non può essere disapprovata.

Possiamo renderla più rilassata sotto alcune condizioni.

- Hold and Wait, richiede che un processo richieda tutte le risorse a lui necessarie in un istante di tempo, e il processo è bloccato finché tutte le richieste possono essere soddisfatte simultaneamente.

È inefficiente: un processo può essere in attesa molto tempo prima che tutte le risorse necessarie siano libere contemporaneamente; un processo può trattenere risorse per molto tempo, anche se non le sta usando.

Un processo non può sapere in anticipo quali risorse gli serviranno.

- No preemption, se a un processo che trattiene certe risorse viene negata una ulteriore richiesta, quel processo deve rilasciare le sue risorse originali e richiederle di nuovo.

Alternativamente l'OS può passare a un secondo processo e richiedere che rilasci le sue risorse.

Un processo deve avere una priorità maggiore dell'altro.

Questo approccio è pratico solamente per risorse per le quali lo stato può essere salvato e più tardi ripristinato (ad esempio la CPU).

- Circular Wait, posso prevenirla definendo un ordinamento lineare dei tipi di risorse.

Se una risorsa di tipo R è stata assegnata ad un processo, il più recente può richiedere solamente risorse di tipo R.

Si ricorre nelle stesse problematiche della hold-and-wait.

- **Avoid Deadlock:** si fanno scelte dinamiche appropriate basate sullo stato corrente dell'allocazione delle risorse.

Deadlock avoidance:

Una decisione viene fatta dinamicamente a prescindere da se la corrente richiesta di allocazione di risorse potrà, se soddisfatta, potenzialmente portare a deadlock.

Richiede conoscenza delle future richieste di processo.

Permette più concorrenza rispetto alla deadlock prevention.

Ci sono due approcci alla deadlock avoidance:

- **Resource Allocation Denial:** non si concede una richiesta di risorse incrementale ad un processo se questa allocazione potrebbe portare a deadlock.
- **Process Initiation Denial:** non si avvia un processo se le sue necessità potrebbero portare a deadlock.

L'OS permette l'esecuzione di un nuovo processo solo se la somma delle richieste di risorse del processo running e le risorse richieste dal nuovo processo è minore delle risorse disponibili:

$$\text{richieste risorse processo running} + \text{richieste risorse nuovo processo} < \text{risorse disponibili}$$

es.

Se P1 e P2 sono in running, P3 e P4 non possono essere eseguite

	R1	R2	R3
	9	3	6

Resource vector R

	R1	R2	R3
P1	3	2	2
P2	6	1	3
P3	3	1	4
P4	4	2	2

Claim matrix C

Safe state: uno stato è safe se esiste una sequenza di allocazione di risorse tale da non risultare in deadlock.

es. Determinazione di un safe state

Stato di un sistema consistente in 4 processi e 3 risorse.

Sono state fatte allocazioni ai 4 processi.

	R1	R2	R3
P1	3	2	2
P2	6	1	3
P3	3	1	4
P4	4	2	2

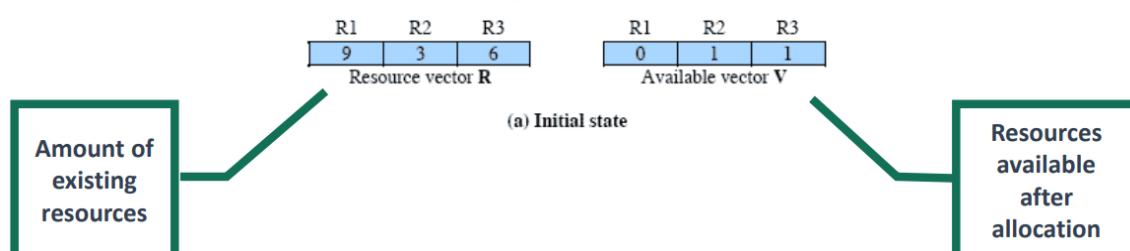
Claim matrix C
matrice delle risorse necessarie ad ogni processo per essere eseguito

	R1	R2	R3
P1	1	0	0
P2	6	1	2
P3	2	1	1
P4	0	0	2

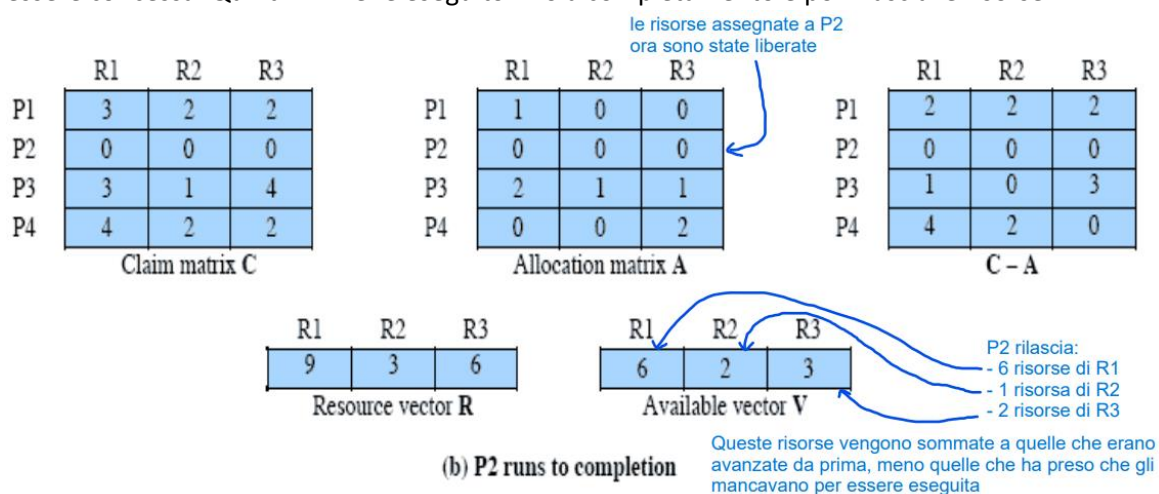
Allocation matrix A
matrice delle allocazioni concesse

	R1	R2	R3
P1	2	2	2
P2	0	0	1
P3	1	0	3
P4	4	2	0

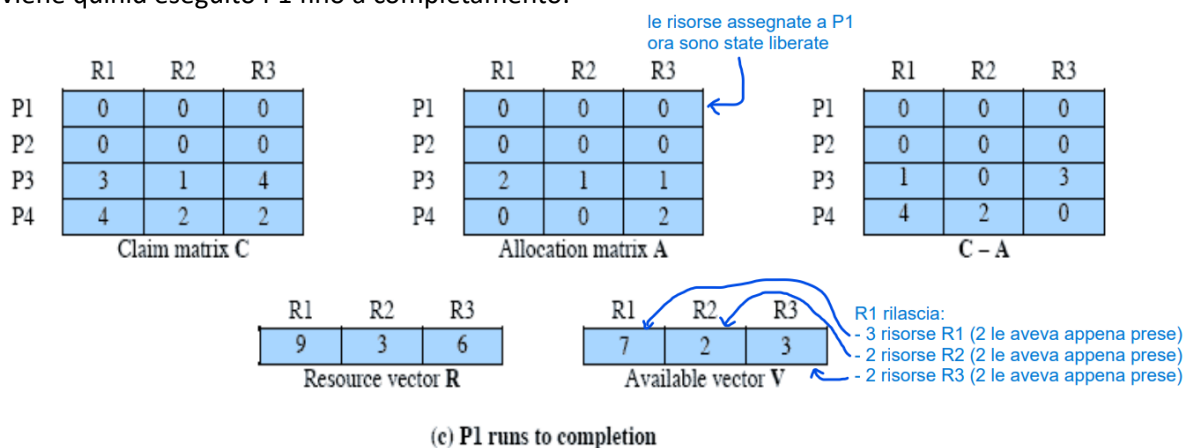
C - A
è la matrice di quello che manca ai processi per essere eseguiti, quindi di quello che potrebbero richiedere



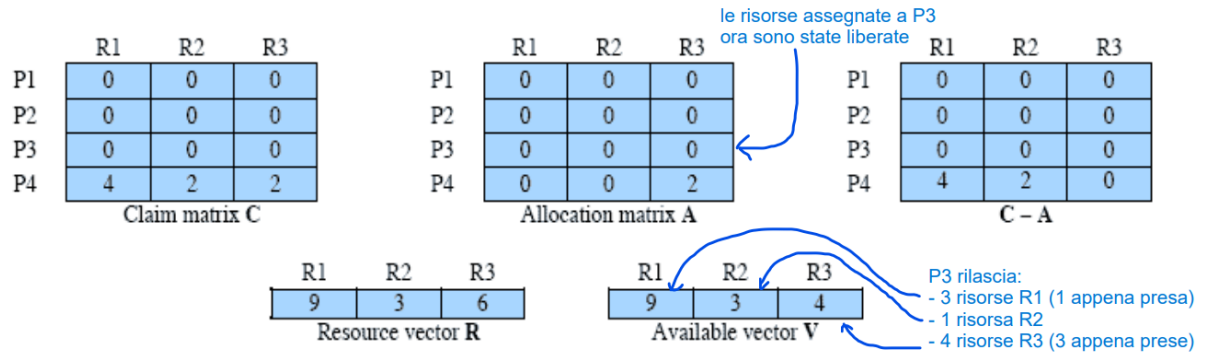
A P2 per essere eseguito serve una risorsa R3. Vediamo dall'available vector che questa può essere concessa. Quindi P2 viene eseguito fino a completamento e poi rilascia le risorse:



Ora a P1 per essere eseguito servono 2 risorse R1, 2 R2 e 2 R3. Prende queste risorse e viene quindi eseguito P1 fino a completamento:



P3 viene eseguito fino a completamento:

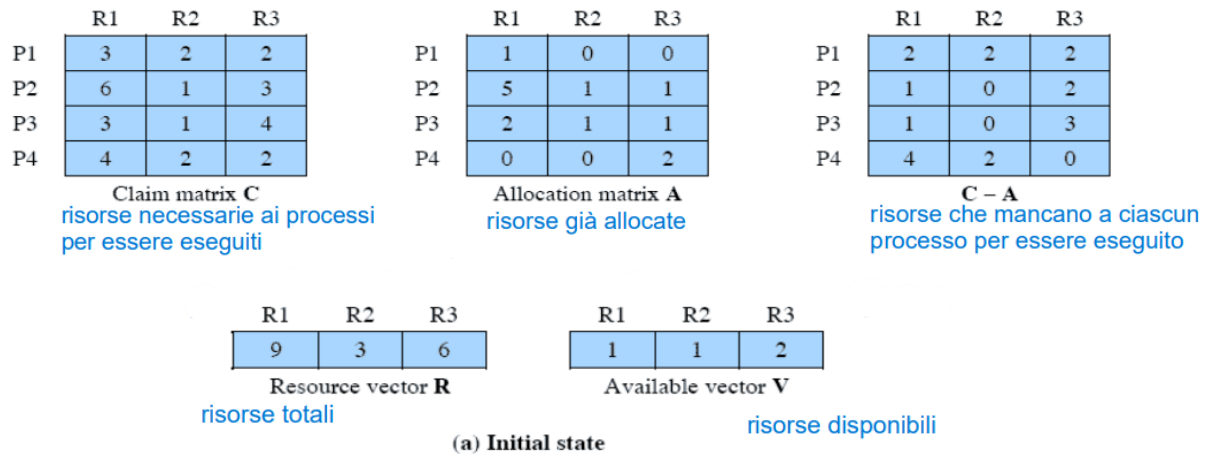


(d) P3 runs to completion

Ora quindi tutte le risorse sono disponibili a P4, che può essere eseguito senza problemi.

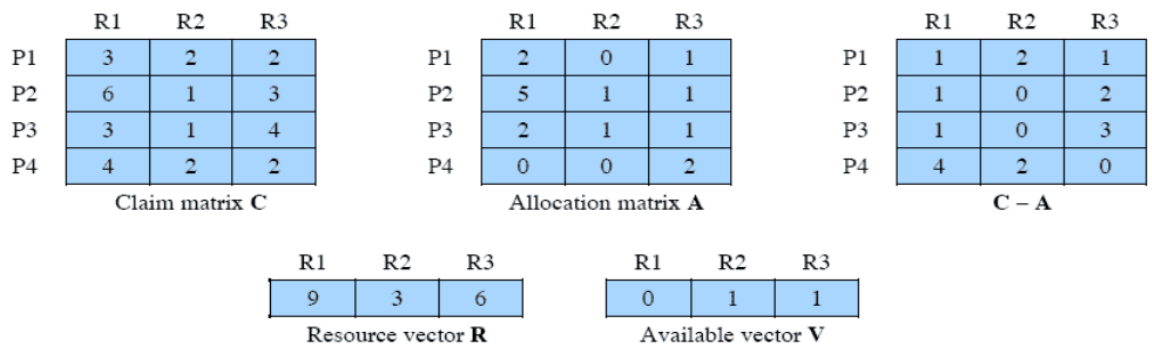
Concludiamo quindi che lo stato originariamente definito è un safe state.

es. Determinazione di un unsafe state



(a) Initial state

P1 richiede 1 risorsa R1 e 1 risorsa R3. Arriviamo quindi allo stato:



(b) P1 requests one unit each of R1 and R3

Questo nuovo stato è safe??

Deadlock avoidance:

```
struct state {  
    int resource[m];  
    int available[m];  
    int claim[n][m];  
    int alloc[n][m];  
}
```

(a) global data structures

```
if (alloc [i,*] + request [*] > claim [i,*])  
    < error >; /* total request > claim*/  
else if (request [*] > available [*]) la quantità di risorse richieste è maggiore  
    < suspend process >; di quelle a disposizione  
else { /* simulate alloc */  
    < define newstate by: si definisce il nuovo stato  
        alloc [i,*] = alloc [i,*] + request [*];  
        available [*] = available [*] - request [*] >;  
    }  
    if (safe (newstate)) si determina se il nuovo stato è safe o no  
        < carry out allocation >;  
    else {  
        < restore original state >;  
        < suspend process >;  
    }  
}
```

(b) resource alloc algorithm

Codice per capire se uno stato è safe: **Banker's algorithm**

```
boolean safe (state S) {  
    int currentavail[m];  
    process rest[<number of processes>];  
    currentavail = available; risorse correntemente disponibili  
    rest = {all processes};  
    possible = true;  
    while (possible) { controllo che ci sia almeno un altro processo che  
        <find a process  $P_k$  in rest such that se mi facesse richieste potrebbe arrivare a termine  
            claim [k,*] - alloc [k,*] <= currentavail;>  
        if (found) { se trova un tale processo /* simulate execution of  $P_k$  */  
            currentavail = currentavail + alloc [k,*];  
            rest = rest - { $P_k$ }; definisce il nuovo stato  
        } poi ricomincia il ciclo trovando un nuovo processo  
        else possible = false; se arriva non trova nessun processo che  
                                possa essere portato a termine si ferma  
    }  
    return (rest == null); True se sono stati tolti tutti i processi dall'insieme rest, quindi se tutti i  
                            processi possono essere portati a termine.  
                            False altrimenti.  
}
```

Vantaggi di deadlock avoidance:

Process initiation denial non è ottimale, mentre invece Resource allocation denial presenta alcuni vantaggi:

- Non è necessario fare preempt e roll back sui processi, come in deadlock detection (discuteremo presto la detection)
- È meno restrittivo rispetto a deadlock prevention

Svantaggi:

- Il massimo necessario delle risorse per ciascun processo deve essere dichiarato a priori
- Processi presi in considerazione devono essere indipendenti e con nessuna necessità di sincronizzazione
- Ci dev'essere un numero fisso di risorse da allocare
- Nessun processo può fare la exit mentre trattiene delle risorse

Strategie di deadlock prevention sono molto conservative, limitano l'accesso alle risorse imponendo restrizioni sui processi.

Strategie di deadlock detection invece fanno l'opposto, le richieste di risorse sono garantite ogni volta possibile

- **Detect Deadlock:** si cerca di individuare la presenza di deadlock e fare in modo di risolvere

Un controllo per deadlock può essere fatto tanto frequentemente quanto ogni risorsa richiesta, o meno frequentemente a seconda di quanto sia probabile che avvenga un deadlock.

Vantaggi di deadlock detection ad ogni richiesta di risorsa:

- Porta a una early detection, l'algoritmo è semplice

Svantaggi:

- Check frequenti consumano un tempo non indifferente della CPU

Algoritmo:

- Segniamo ogni processo che non ha nessuna risorsa allocata (se non ha nessuna risorsa non può partecipare in un deadlock)
- Cerchiamo un processo P_i che non è segnato e tale che il numero ogni risorsa necessaria a P_i (la i -esima riga della matrice delle richieste Q) sia minore o uguale del numero delle risorse disponibili corrispondenti (allocation vector).
- Se non troviamo nessun processo del genere terminiamo l'algoritmo.
- Se troviamo un processo del genere, segniamo il processo i trovato, e sommiamo la riga corrispondente nella matrice delle risorse allocate A all'allocation vector (gli assegnamo le risorse che avanzavano nell'allocation vector, così che possa essere eseguito). Torniamo al secondo punto

es.

	R1	R2	R3	R4	R5
P1	0	1	0	0	1
P2	0	0	1	0	1
P3	0	0	0	0	1
P4	1	0	1	0	1

Request matrix Q
matrice delle risorse necessarie a ciascun processo

	R1	R2	R3	R4	R5
P1	1	0	1	1	0
P2	1	1	0	0	0
P3	0	0	0	1	0
P4	0	0	0	0	0

Allocation matrix A
matrice delle risorse allocate

R1	R2	R3	R4	R5
2	1	1	2	1

Resource vector
risorse complessive

R1	R2	R3	R4	R5
0	0	0	0	1

Allocation vector
risorse che avanzano

Segniamo P4 perché non ha nessuna risorsa allocata.

Segniamo P3 perché se gli diamo le risorse che avanzano può terminare e rilasciare le risorse.

Anche con le risorse appena liberate da P3, né P1 né P2 possono terminare

→ deadlock

Alcuni approcci di deadline detection-recovery, in ordine crescente di sofisticatezza:

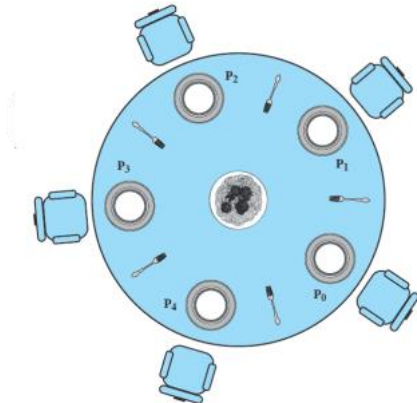
- Abolisce tutti i processi coinvolti nel deadlock (più comune)
- Fa back up di ogni processo nel deadlock a un checkpoint definito precedentemente e restarta ogni processo
- Si aboliscono successivamente i processi nel deadlock finché il deadlock non c'è più
- Si fa preempt delle risorse successivamente finché il deadlock non c'è più (richiede un meccanismo di rollback)

Tabella riassuntiva degli approcci al deadlock:

Approach	Resource Allocation Policy	Different Schemes	Major Advantages	Major Disadvantages
Prevention	Conservative; undercommits resources	Requesting all resources at once hold and wait	• Works well for processes that perform a single burst of activity • No preemption necessary	• Inefficient • Delays process initiation • Future resource requirements must be known by processes
		Preemption no preemption	• Convenient when applied to resources whose state can be saved and restored easily	• Preempts more often than necessary
		Resource ordering circular wait	• Feasible to enforce via compile-time checks • Needs no run-time computation since problem is solved in system design	• Disallows incremental resource requests
Avoidance	Midway between that of detection and prevention	Manipulate to find at least one safe path	• No preemption necessary	• Future resource requirements must be known by OS • Processes can be blocked for long periods
Detection	Very liberal; requested resources are granted where possible	Invoke periodically to test for deadlock	• Never delays process initiation • Facilitates online handling	• Inherent preemption losses

3. PROBLEMA DEI 5 FILOSOFI

Problema dei 5 filosofi:



Nessuna coppia di filosofi può usare la stessa forchetta allo stesso tempo (mutua esclusione). Nessun filosofo deve morire di fame (avoid deadlock e starvation).

Questo problema è rilevante in quanto illustra problemi di base di deadlock e starvation, è uno standard test case per valutare approcci alla sincronizzazione.

Soluzione usando N=5 semafori:

```
semaphore fork [5] = {1};  
void philosopher (int i)  
{  
    while (true) {  
        think();  
        wait (fork[i]);  
        wait (fork [(i+1) mod 5]);  
        eat();  
        signal(fork [(i+1) mod 5]);  
        signal(fork[i]);  
    }  
}
```

aspetta che si liberino le forchette i e i+1
mangia
rilascia le forchette

Prova a prendere la i-esima forchetta a sinistra, poi la (i+1)-esima a destra. Questa soluzione però porta facilmente a deadlock.

Soluzione corretta:

tolgo una sedia, ossia lascio spazio solo per 4 filosofi alla volta. Se ho 4 filosofi e 5 forchette sicuramente uno riuscirà a mangiare. Quando qualcuno finisce di mangiare allora si siede il filosofo che era rimasto fuori.

```

semaphore fork[5] = {1};
semaphore room = {4};
void philosopher (int i)
{
    while (true) {
        think();
        wait (room); prova a prendere una sedia
        wait (fork[i]); prova a prendere forchetta sinistra
        wait (fork [(i+1) mod 5]); prova a prendere forchetta destra
        eat(); mangia
        signal (fork [(i+1) mod 5]);
        signal (fork[i]); lascia le forchette e la sedia
        signal (room);
    }
}

```

La mutua esclusione può essere implementata anche a livello del software.

Supponiamo di avere processi che comunicano tramite una memoria centrale su una macchina sigle/multi-processor, e assumiamo di avere mutua esclusione al livello di accessi alla memoria (lettura/scrittura arriverà sempre in mutua esclusione, non ci saranno processi diversi che operano sulla stessa locazione in memoria contemporaneamente).

Le operazioni di read/write per la stessa locazione sono disposte secondo un ordine da una specie di memory arbiter (l'ordine è sconosciuto).

Nessun supporto nell'OS, hardware o programming language.

4. ALGORITMI DI DEKKER E DIJKSTRA

Algoritmo Dekker: algoritmo per la mutua esclusione, progettato da Dekker e riportato da Dijkstra

Primo tentativo:

```

/* global */
int turn = 0;

// assignments valid for P0 (flip for P1)
int me = 0, other = 1;
while (true) {
    /*NCS*/
    while (turn != me) busy waiting finché non è il mio turno
        /* busy wait */ ;
    /* CS */ uscito dal ciclo (è il mio turno): sezione critica
    turn = other; passo il turno a qualcun'altro
}

```

Funziona perfettamente, ma comporta poca alternanza. Se io dovessi entrare una volta al secondo nella zona critica, e l'altro processo ogni mezz'ora, io comunque entrerei ogni mezz'ora. Non potrei comunque accederci.

Secondo tentativo:

```
/* global */
boolean flag[2] = {false, false};

// assignments valid for P0 (flip for P1)
int me = 0, other = 1;

while (true) {
    /*NCS*/
    while (flag[other]) busy waiting, aspetto finché l'altro è interessato alla sezione critica, quando non è più interessato entro io
        /* busy wait */ ;
    flag[me] = true; sono interessato
    /* CS */ sezione critica
    flag[me] = false; non sono più interessato
}
```

Uso due flag per capire se l'altro processo è interessato a entrare nella critical section o meno. Questa soluzione però non garantisce sempre mutua esclusione: due processi in parallelo potrebbero vedersi entrambi non interessati, e quindi porsi entrambi interessati ed entrare entrambi nella critical section.

Terzo tentativo:

```
// assignments valid for P0 (flip for P1)
int me = 0, other = 1;

while (true) {
    /*NCS*/
    flag[me] = true; sono interessato
    while (flag[other]) busy waiting, aspetto finché l'altro è interessato alla sezione critica, quando non è più interessato entro io
        /* busy wait */ ;
    /* CS */ sezione critica
    flag[me] = false; non sono più interessato
}
```

segnalo subito che voglio entrare nella sezione critica.

Questa soluzione può però portare a deadlock (sia io che un altro processo siamo interessati, e stiamo aspettando che l'altro non sia più interessato).

Quarto tentativo:

```
// assignments valid for P0 (flip for P1)
int me = 0, other = 1;

while (true) {
    /*NCS*/
    flag[me] = true; sono interessato
    while (flag[other]) {
        flag[me] = false; finché l'altro è interessato dico che non sono interessato, aspetto un pochino e ci riprovo di nuovo
        /* delay */
        flag[me] = true;
    }
    /* CS */ sezione critica
    flag[me] = false; non sono più interessato
}
```

Soluzione corretta: Dekker's Algorithm

Aspetto che l'altro processo entri in busy waiting io esco dal ciclo while

```
/* global */
int turn = 0;

int me = 0, other = 1; // P0 (flip for P1)

while (true) {
    /*NCS*/
    flag[me] = true; sono interessato
    while (flag[other]) { finché l'altro è interessato
        if (turn == other) { se è il turno dell'altro
            flag[me] = false; dico che non sono interessato
            while (turn == other) /* busy wait */ ;
            flag[me] = true; aspetto che l'altro processo dica che è il mio turno
        }
    }
    /* CS */ sezione critica
    turn = other; è il turno dell'altro
    flag[me] = false; non sono più interessato
}
```

La soluzione di Dekker funziona per due processi. Per estenderla a più processi dovremmo modificarla un po'.

Dijkstra ha generalizzato l'algoritmo di Dekker per gestire N entità.

Ha anche fornito definizioni per le proprietà di mutua esclusione (ME), no deadlock (ND) e no starvation (ND). Da notare che NS implica ND

Algoritmo di Dijkstra: generalizzazione dell'algoritmo di Dekker per N entità

```
/* global storage */
boolean interested[N] = {false, ..., false}
boolean passed[N]      = {false, ..., false}
int k = <any>           // k ∈ {0, 1, ..., N-1}

/* local info */
int i = <entity ID>     // i ∈ {0, 1, ..., N-1}
```

Assumeremo che le operazioni di modifica di "k" sono atomiche, in quanto ci possono scrivere tutti.

```
while (true) {
    /*NCS*/
    1. interested[i] = true io sono interessato
    2. while (k != i) { finché è il turno di qualcun'altro
    3.     passed[i] = false non sono passato
    4.     if (!interested[k]) then k = i se anche il processo k è interessato continuo ad aspettare, se il processo k non è interessato allora dico k=i
        Mi prendo quindi il turno
    5.     passed[i] = true sono passato
    6.     for j in 1 ... N except i do se c'è anche un solo altro processo che è passato allora ricomincio da capo
    7.         if (passed[j]) then goto 2
    8.     <critical section> sezione critica
    9.     passed[i] = false; interested[i] = false
}
```

C'è il rischio che più di un processo superi il primo ciclo while, e che entrambi settino passed a true. Se più processi sono passati, controlleranno la condizione del while, ma uno dei processi sarà stato l'ultimo a modificare k, che troverà uguale al proprio i, quindi andrà di nuovo alla riga 5.

È possibile che un processo debba tornare indietro più volte.

Con questo metodo ho la certezza che uno solo andrà alla critical section.

es. $N \geq 4$ (inizialmente $k=4$)

P1	P2	P3
intr[1] = true <i>P1 interessato</i>		
while (k!=1)		
pass[1] = false		
	intr[2] = true <i>P2 interessato</i>	
	while (k!=2)	
	pass[2] = false	
		intr[3] = true <i>P3 interessato</i>
		while (k!=3)
		pass[3] = false
if (!intr[4]) <i>P4 non è interessato, quindi pongo k=1</i>		
	if (!intr[4]) <i>P4 non è interessato, quindi pongo k=2</i>	
		if (!intr[4]) <i>P4 non è interessato, quindi pongo k=3</i>
k = 1		
	k = 2	
		k = 3

I processi P1, P2 e P3 ora sono pronti ad eseguire le righe 5-7. Solamente uno di loro però sarà in grado di accedere alla critical section.

L'ultima modifica che è stata fatta su di k è k=3, ma questo non implica che k andrà avanti per prima, k viene usata solo per risolvere conflitti, quindi lo scheduler potrebbe anche far passare P1 o P2. Si crea un conflitto quando due processi settano entrambi passed[i]=true prima che l'altro sia entrato e poi uscito dalla sezione critica.

Un possibile continuazione senza conflitti è la seguente: lo scheduler fa eseguire P3 con P1 e P2 ancora fermi alla riga 5.

P3
pass[3] = true
for j in 1..N except 3 do
if pass[j] goto 2 (<i>skipped</i>)
<critical section>
pass[3] = false
intr[3] = false

P3 ha finito. C'è conflitto fra P1 e P2?

P1	P2
while (k!=1)	
pass[1] = false	
	while (k!=2)
	pass[2] = false
if (!intr[3]) (<i>true: P3 done</i>)	if (!intr[3]) (<i>true: P3 done</i>)
then k = 1	
while (k!=1) (<i>skipped</i>)	
pass[1] = true	
	then k = 2
	while (k!=2) (<i>skipped</i>)
if (pass[2]) goto 2 (<i>taken!!</i>)	if (pass[1]) goto 2 (<i>taken!!</i>)

Quando si verifica un conflitto l'algoritmo torna indietro alla riga 2 e cancella passed[i], eccetto per il processo che ha settato k per ultimo.

All'esame c'è o una domanda sull'algoritmo di Dijkstra o sull'algoritmo del panettiere. Ci farà vedere l'algoritmo ma con un errore.